Stage 1: Find and Critique a Dataset

1.1 Dataset Selection and Description

The dataset chosen for this project is titled **"Government Expenditure on R&D by Sub-Sector of Economic Activity during 2005-06 to 2009-10"**, sourced from data.gov.in, India's official open government data platform. It presents annual R&D spending (in crores of rupees) across various **economic sub-sectors**, grouped under broader **sectors**, over a 5-year period.

Link to the dataset:
https://www.data.gov.in/resource/government-expenditure-rd-sub-sector-economic-activity-during-2005-06-2009-10

Each record in the dataset corresponds to a specific sub-sector and contains:

- The **sector** name,
- The **sub-sector or economic activity**,
- And the **expenditure figures** from 2005–06 through 2009–10.

The dataset is provided in **CSV format**, which is well-suited for relational transformation and easy integration into MySQL.

1.2 Dataset Assessment

Following the criteria from discussion 1.104, the dataset has been evaluated as follows:

Quality

The information comes from a reliable government website and shows real R&D spending data from official sources.

Detail

There are 47 rows in the dataset, which cover a number of sectors and sub-sectors. It doesn't have organizational-level granularity, but it does have detailed breakdowns by economic activity and across multiple fiscal years. This is enough detail for trend analysis and sectoral comparison.

Documentation

The portal page has some basic metadata about the dataset, like its title, description, and how often it is updated. Even though there isn't a clear data dictionary, the column names are clear, and the terms for sectors and sub-sectors make it easy to understand.

Interrelation

The original dataset is flat, but it can be broken down into three related groups: **Sectors**, **Sub-Sectors**, and **Expenditure** by Year. Because of these natural connections, it is very good for E/R modeling.

Use

You can do a lot of things with this dataset, like keep track of investment trends, figure out the country's research and development priorities, or look at how different sectors are growing. The data is only from 2005 to 2010, though, and it doesn't include geographic breakdowns (like state-level data), which makes it harder to do spatial analysis..

Discoverability

Using the search and category filters on data.gov.in made it pretty easy to find the dataset. It was in the "Science and Technology" and "Industry" sections. There were a number of other datasets available, but not many of them had both time-series data and sectoral classification in a format that could be used.

1.3 Terms of Use and Licensing

The dataset is governed by the **Open Government Data License – India**. The license:

- Allows unrestricted reuse, adaptation, and redistribution.
- Requires attribution to the source (data.gov.in and the original ministry).
- Prohibits misrepresentation or distortion of the data.

The "Catalog Info" page on the data.gov.in portal had clear information about the dataset's license and attribution. The licensing terms are clear and allow for reuse.

1.4 Interest in the Dataset

This dataset is interesting because it shows what India has historically prioritized in terms of research and development, which is an important part of national growth. Knowing which sectors got the most R&D money can help us understand economic strategy, industrial policy, and how science is moving forward.

In addition, the dataset strikes a good balance between being rich enough for modeling and not being pre-normalized, which makes it possible to design a real database. It also has time-series data, which lets you do longitudinal analysis, something that isn't common in many open datasets.

1.5 Research Questions

To look at the dataset using a relational database approach, the following research questions were made:

1. **Which subsectors received the highest total R&D spending across all years?**
   → This requires aggregation of `expenditure` by `subsector_id` across multiple records.

2. **How has R&D spending changed over time for each major sector?**
   → This involves joining `Sector`, `SubSector`, and `Expenditure` tables, grouped by year and sector.

3. **Which year had the highest overall R&D spending across all sectors?**
   → This question utilizes the normalized `Expenditure` table to compare annual totals.

You can't answer these questions in a spreadsheet because they need joins, filters, and aggregations. They can also be used in a Node.js + MySQL web app, which is what you need to do for Stage 4 of the midterm.

Stage 2: Model your data

2.1 Transforming the Dataset for Relational Implementation

The original dataset from data.gov.in had five years' worth of R&D spending (2005–2010) in separate columns. This format is common in spreadsheets, but it doesn't work for designing a relational database.

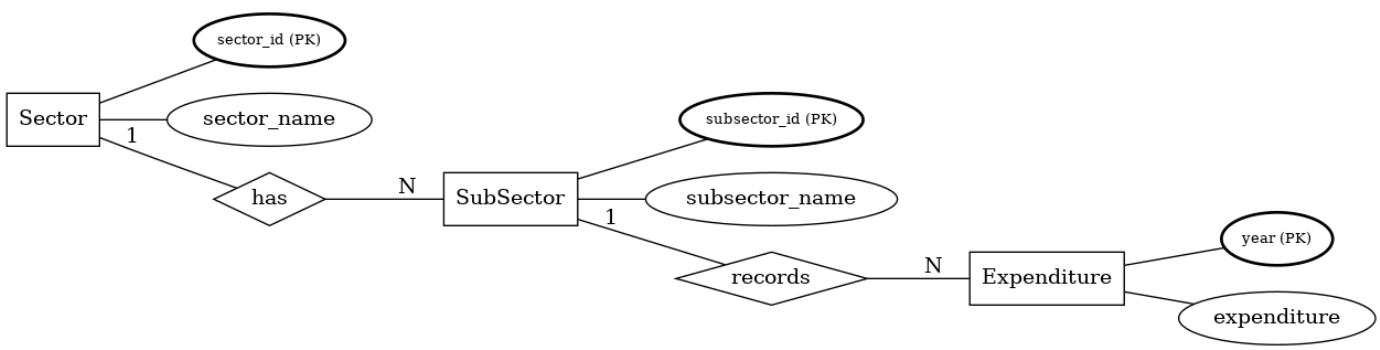
We changed the data from a wide format to a long format so that it would follow normalization rules and allow for flexible year-based queries. This process turned each column with a year into a single Year field and moved the value for each year's spending into a single Expenditure field. Now, each row shows one specific piece of information: the amount spent on research and development in a certain sub-sector in a certain year..

We used **Python and the Pandas library** to do this transformation because it made it easy to change the shape of the data in a way that could be repeated. The following method was used to do the pivoting:

```
df.melt(

    id_vars=["Sectors", "Sub Sectors and Economic Activity"],

    value_vars=["2005-06", "2006-07", "2007-08", "2008-09",
"2009-10"],

    var_name="Year",

    value_name="Expenditure"

)
```

The output was saved as a new CSV `pivoted_dataset.csv` file which will be used to populate the **Expenditure** table in MySQL.

2.2 E/R model of the data with Cardinality



The relational schema has three tables, one for each entity in the E/R model.

Sector

- `sector_id` (Primary Key) — unique identifier for each sector
- `sector_name` — name of the sector (e.g. Agriculture, Manufacturing)

SubSector

- `subsector_id` (Primary Key) — unique identifier for each subsector

- `subsector_name` — name of the subsector (e.g. Fishing, Coal Mining)
- `sector_id` (Foreign Key) — references `Sector(sector_id)`

Expenditure

- `subsector_id` (Foreign Key) — references `SubSector(subsector_id)`
- `year` (Primary Key component) — fiscal year (e.g. 2005–06)
- `expenditure` — R&D expenditure for that subsector in the given year

The primary key for this table is a composite of `subsector_id` and `year`.

Normalization

The database design satisfies the requirements of **Third Normal Form (3NF)**:

- **1NF**: All fields contain atomic values; there are no repeating or multi-valued fields.
- **2NF**: Every non-key attribute in each table is fully dependent on the whole primary key. In `Expenditure`, for example, `expenditure` depends on both `subsector_id` and `year`.
- **3NF**: There are no transitive dependencies; non-key attributes depend only on their table's primary key.

Beyond 3NF

There is no more normalization done. The schema already meets the BCNF requirements because all of the functional dependencies have candidate keys as their determinants. 4NF doesn't apply here because there are no multi-valued dependencies.

Stage 3: Create the Database

3.1 Build the Database Structure

The database was created in MySQL using the Coursera lab environment, which provides a cloud-based VS Code interface with terminal access. The schema is based on the E/R model made in Stage 2 and shows three entities: `Sector`, `SubSector`, and `Expenditure`.

I began by launching the MySQL server in the terminal using:

```
mysql -u root -p
```

After creating and switching to a new database:

```
CREATE DATABASE rnd_project;
USE rnd_project;
```

I then executed the following `CREATE TABLE` statements to define the database schema:

```
CREATE TABLE Sector (
   sector_id INT PRIMARY KEY,
   sector_name VARCHAR(255)
);

CREATE TABLE SubSector (
   subsector_id INT PRIMARY KEY,
   subsector_name VARCHAR(255),
   sector_id INT,
   FOREIGN KEY (sector_id) REFERENCES Sector(sector_id)
);

CREATE TABLE Expenditure (
   subsector_id INT,
   year VARCHAR(10),
   expenditure FLOAT,
   PRIMARY KEY (subsector_id, year),
   FOREIGN KEY (subsector_id) REFERENCES SubSector(subsector_id)
);
```

These tables implement the normalized structure in 3NF, as described in Stage 2.

3.2 Enter Instance Data

I wrote a Python script to automate the process of adding data instead of writing over 200 INSERT statements by hand. The script reads the pivoted_dataset.csv file, which has R&D spending by year and subsector, and then does these things:

- Cleans the data by removing unwanted rows that contain labels such as “Total”, “GDP”, or “Expenditure”
- Strips formatting prefixes like numbering or bracketed letters (`(A)`, `(B)`) from sector and subsector names
- Assigns unique IDs to each sector and subsector
- Keeps the correct relationship between sectors and subsectors
- Aggregates expenditure data by (subsector_id, year)

- Generates SQL `INSERT INTO` statements for all three normalized tables (`Sector`, `SubSector`, `Expenditure`)
- Outputs the full set of SQL commands to a `.sql` file, which I can simply copy paste in the MySQL terminal within the Coursera lab environment.

Python Script:

```
import pandas as pd
import re

# Load dataset
df = pd.read_csv("pivoted_dataset.csv")
df.columns = ['sector_name', 'subsector_name', 'year', 'expenditure']

# Function to clean prefixes like (A), (1.), etc.
def clean_prefix(text):
    return re.sub(r'^\(?[A-Za-z0-9]+[).]?\s*', '', str(text).strip())

# Clean names
df['sector_name'] = df['sector_name'].apply(clean_prefix)
df['subsector_name'] = df['subsector_name'].apply(clean_prefix)

# Filter out unwanted rows
exclusion_keywords = ['Total', 'GDP', 'Expenditure']
df_cleaned = df[
    ~df['sector_name'].str.contains('|'.join(exclusion_keywords), case=False, na=False) &
    ~df['subsector_name'].str.contains('|'.join(exclusion_keywords), case=False, na=False)
].copy()

# Assign unique IDs
sector_lookup = {name: idx + 1 for idx, name in
enumerate(sorted(df_cleaned['sector_name'].unique()))}
df_cleaned['sector_id'] = df_cleaned['sector_name'].map(sector_lookup)

subsector_pairs = df_cleaned[['subsector_name', 'sector_id']].drop_duplicates()
subsector_lookup = {name: idx + 1 for idx, name in
enumerate(sorted(subsector_pairs['subsector_name'].unique()))}
df_cleaned['subsector_id'] = df_cleaned['subsector_name'].map(subsector_lookup)

# Group by subsector and year
df_grouped = df_cleaned.groupby(['subsector_id', 'year'], as_index=False)['expenditure'].sum()

# Escape strings for SQL safety
def escape_sql_string(value):
    return str(value).replace("'", "''")
```

```
# Create SQL INSERT statements
sector_inserts = [
    f"INSERT INTO Sector (sector_id, sector_name) VALUES ({sid},
'{escape_sql_string(name)}');"
    for name, sid in sector_lookup.items()
]

subsector_inserts = [
    f"INSERT INTO SubSector (subsector_id, subsector_name, sector_id) VALUES "
    f"({subsector_lookup[row.subsector_name]}, '{escape_sql_string(row.subsector_name)}',
{row.sector_id});"
    for _, row in subsector_pairs.iterrows()
]

expenditure_inserts = [
    f"INSERT INTO Expenditure (subsector_id, year, expenditure) VALUES "
    f"({row.subsector_id}, '{row.year}', {row.expenditure});"
    for _, row in df_grouped.iterrows()
]

# Write output to SQL file
with open("load_data_statements_cleaned_final_grouped.sql", "w") as f:
    f.write("-- Sector\n" + '\n'.join(sector_inserts) + "\n\n")
    f.write("-- SubSector\n" + '\n'.join(subsector_inserts) + "\n\n")
    f.write("-- Expenditure\n" + '\n'.join(expenditure_inserts) + "\n")
```

3.3 Reflection on Database Accuracy

The structure does a good job of showing the data. Foreign keys and joins keep the relationships between sectors, subsectors, and expenditures correct. The composite key in the Expenditure table makes sure that there is one record for each subsector every year, just like the source data shows.

Using subsector_id and year as a composite primary key worked really well. It made it easy to write queries for time-series comparisons without needing a surrogate ID.

The only small problem was dealing with special characters (like apostrophes) in sector names, but this was fixed by using Python's built-in SQL string escaping.

3.4 List of SQL Commands and Corresponding Research Questions

I wrote the following SQL queries to answer the research questions that came up in Stage 1. After the data was cleaned and put into the normalized database, each query was run in the MySQL environment.

Question 1:

Which subsectors received the highest total R&D spending across all years?

```
SELECT ss.subsector_name, SUM(e.expenditure) AS total_spending
FROM SubSector ss
JOIN Expenditure e ON ss.subsector_id = e.subsector_id
GROUP BY ss.subsector_id, ss.subsector_name
ORDER BY total_spending DESC
LIMIT 10;
```

It gives you the top 10 subsectors that spent the most on research and development over the past five years.

Question 2:

How has R&D spending changed over time for each major sector?

```
SELECT s.sector_name, e.year, SUM(e.expenditure) AS yearly_spending
FROM Sector s
JOIN SubSector ss ON s.sector_id = ss.sector_id
JOIN Expenditure e ON ss.subsector_id = e.subsector_id
GROUP BY s.sector_name, e.year
ORDER BY s.sector_name, e.year;
```

It shows how much money was spent in each sector each year, which lets you look at trends in each sector over time.

Question 3:

Which year had the highest overall R&D spending across all sectors?

```
SELECT year, SUM(expenditure) AS total_spending
FROM Expenditure
GROUP BY year
ORDER BY total_spending DESC
LIMIT 1;
```

It gives back the year with the most money spent on R&D. The result shows which year had the most investment activity.

Summary

All three research questions identified in Stage 1 were fully answerable using standard SQL operations (`JOIN`, `GROUP BY`, `SUM`, `ORDER BY`).

Stage 4. Create a Simple Web Application

4.1 Application Overview

I made a simple web app with Node.js and the Express framework to answer the research questions that came up in Stage 1. The app connects to the normalized MySQL database that was made in Stage 3 and shows results through a browser interface that users can interact with.

The application runs locally on port 3000 and exposes three routes:

- `/top-subsectors` – shows top R&D spending subsectors
- `/sector-trends` – displays yearly R&D spending by major sector
- `/top-year` – identifies the year with the highest total R&D investment.

4.2 Database Interaction

The app uses the `mysql2` Node.js library to connect to the database. A separate `db.js` module handles the connection using credentials for the local MySQL user. For example:

```
// db.js

const mysql = require('mysql2');

const connection = mysql.createConnection({

  host: 'localhost',

  user: 'root',

  password: '',

  database: 'rnd_project'
```

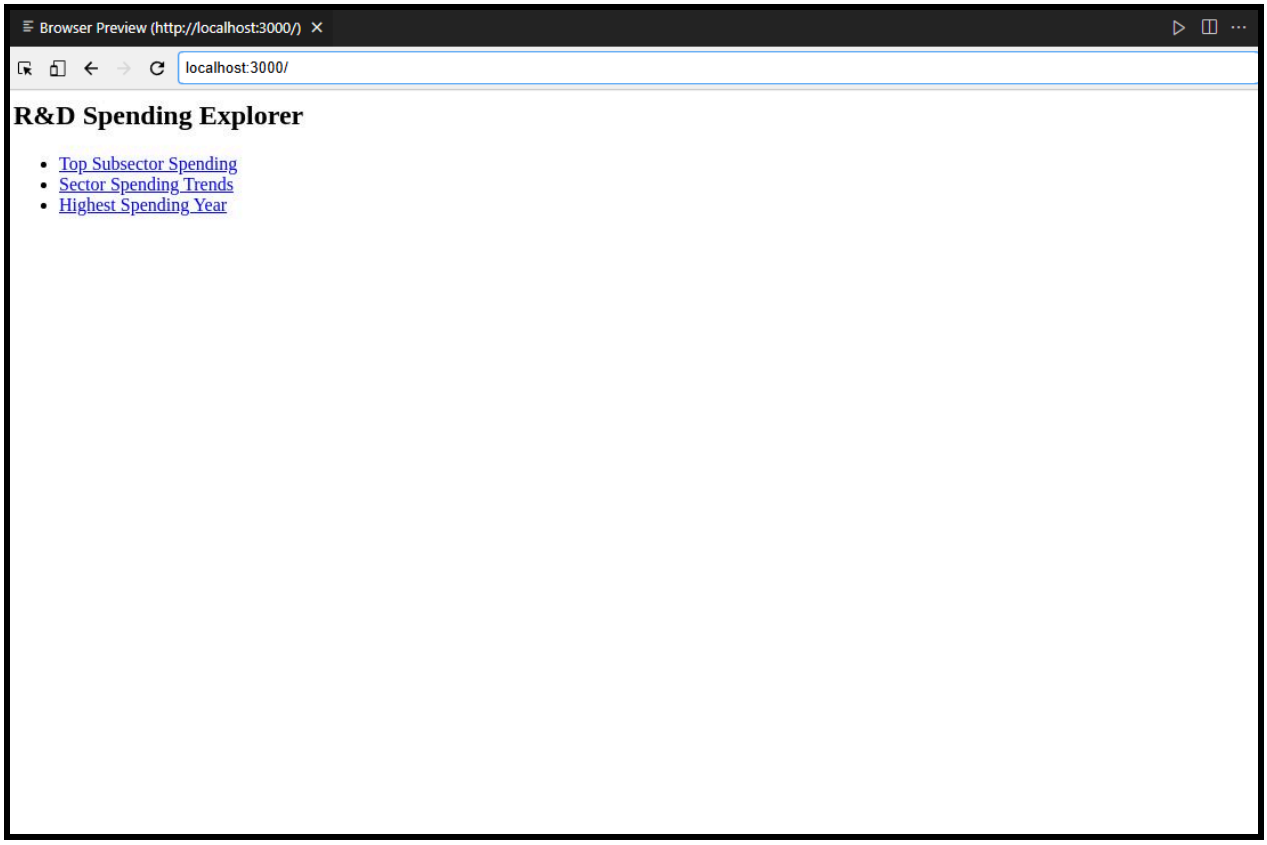
```
});
```

The queries return HTML-formatted responses.

4.3 Application Functionality and Presentation

The app's main entry point (/) has a list of questions that you can use to get around. Each linked route runs a SQL query and shows the results in a clear, easy-to-read way. An example of HTML output is a list view of subsector names with spending amounts or a breakdown of spending by sector by year.

Below are screenshots of the output from each route to show that the rendering and query results are correct:



Browser Preview (http://localhost:3000/top-subsectors)

localhost:3000/top-subsectors

Top Subsector Spending

- by industry of origin (at factor cost Rs. Crores): ₹19742201.00
- Public Administration and Defence: ₹33023.22
- Service Sector: ₹31706.74
- Agriculture, Hunting and Related Service Activities: ₹24579.94
- Electricity, Gas, Steam and Water Supply: ₹11293.85
- Drugs & Pharmaceuticals: ₹4277.26
- Health and Social Work: ₹4249.88
- Education: ₹3047.34
- Others Please Specify: ₹2710.89
- Forestry, Logging and Related Service Activities: ₹1949.92

Back

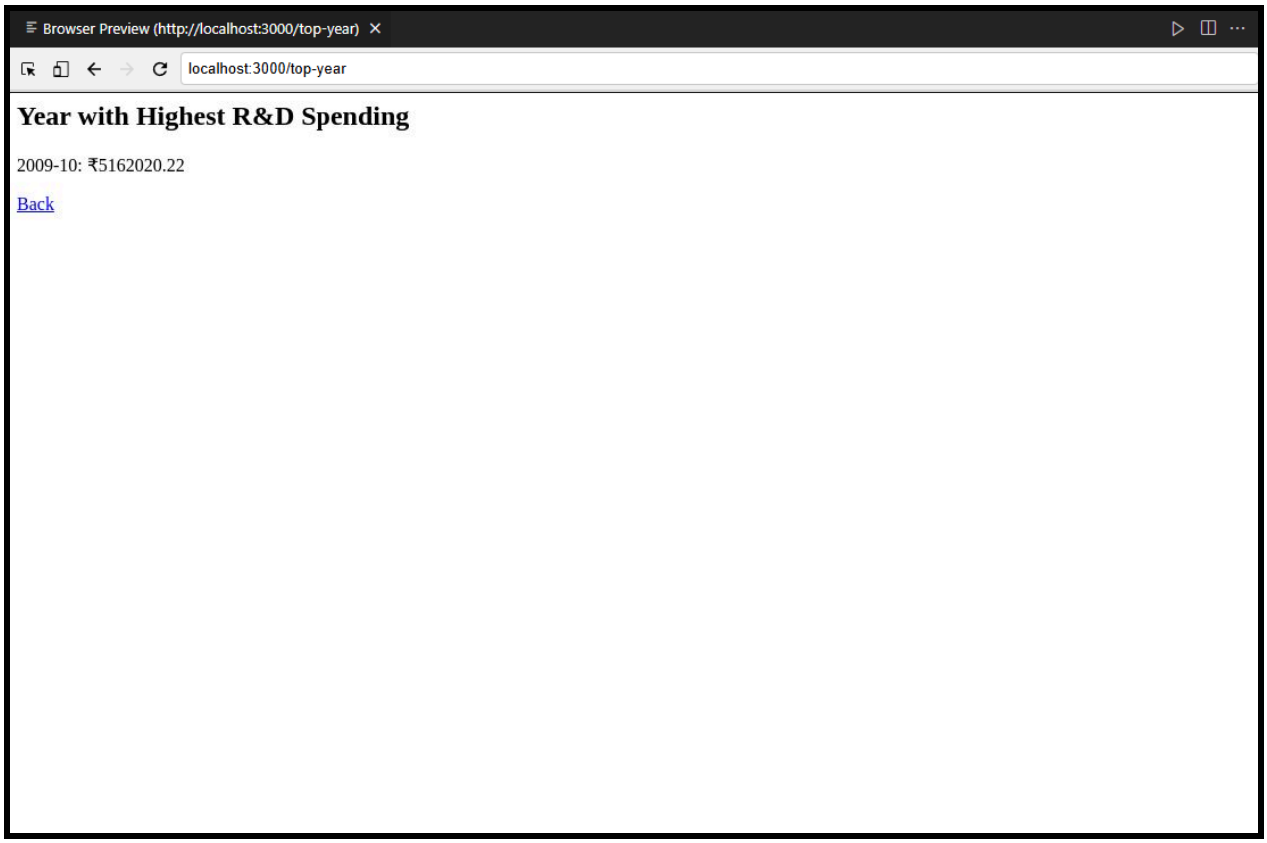
Browser Preview (http://localhost:3000/sector-trends)

localhost:3000/sector-trends

Sector Spending Over Time

- 2005-06 – Agriculture, Forestry & Fishing, Mining & Quarrying: ₹2901643.29
- 2006-07 – Agriculture, Forestry & Fishing, Mining & Quarrying: ₹3371665.80
- 2007-08 – Agriculture, Forestry & Fishing, Mining & Quarrying: ₹3895761.62
- 2008-09 – Agriculture, Forestry & Fishing, Mining & Quarrying: ₹4464625.63
- 2009-10 – Agriculture, Forestry & Fishing, Mining & Quarrying: ₹5136346.10
- 2005-06 – Manufacturing, Construction, Electricity Gas & Water Supply: ₹2557.65
- 2006-07 – Manufacturing, Construction, Electricity Gas & Water Supply: ₹3023.88
- 2007-08 – Manufacturing, Construction, Electricity Gas & Water Supply: ₹3251.29
- 2008-09 – Manufacturing, Construction, Electricity Gas & Water Supply: ₹4256.16
- 2009-10 – Manufacturing, Construction, Electricity Gas & Water Supply: ₹5500.51
- 2005-06 – Public Administration & Defence and other Services: ₹11675.09
- 2006-07 – Public Administration & Defence and other Services: ₹12348.32
- 2007-08 – Public Administration & Defence and other Services: ₹14043.75
- 2008-09 – Public Administration & Defence and other Services: ₹17273.73
- 2009-10 – Public Administration & Defence and other Services: ₹19397.18
- 2005-06 – Transport & Communication: ₹444.40
- 2006-07 – Transport & Communication: ₹487.21
- 2007-08 – Transport & Communication: ₹538.76
- 2008-09 – Transport & Communication: ₹661.89
- 2009-10 – Transport & Communication: ₹776.43

Back



4.4 Motivation and Goal Fulfillment

The web app directly helps with and answers the questions that were found in Stage 1:

- **Which subsectors received the highest total R&D spending across all years?**
- **How has R&D spending changed over time for each major sector?**
- **Which year had the highest overall R&D spending across all sectors?**

The app has set up each of these as a separate endpoint. The output is useful for exploratory analysis and for researchers in government or policy who are looking at patterns in R&D spending.

5. References

Open Government Data (OGD) Platform India.
R&D Statistics - Expenditure by Sector and Subsector
https://data.gov.in

Coursera – Databases and Advanced Data Techniques
University of London / Goldsmiths

Video modules, and lab content covering database normalization, ER modeling, SQL, and web integration.

MySQL Documentation.
The official MySQL Reference Manual
https://dev.mysql.com/doc/

Node.js Documentation.
Node.js v14+ API Reference
https://nodejs.org/en/docs/

Express.js Framework Guide.
Express Application Guide
https://expressjs.com/

W3Schools SQL Tutorial.
Basic SQL query patterns and syntax examples
https://www.w3schools.com/sql/